

《计算机视觉》实验报告

姓名：周梓俊 学号：23120899

实验 5

一、任务 1

a) 核心代码

```
1 # 生成 Haar 特征池并随机选择一部分参与训练
2 full_feature_pool = generate_feature_pool(
3     CONFIG["window_size"],
4     CONFIG["feature_stride"],
5     CONFIG["feature_size_step"],
6 )
7 selected_features = (
8     rng.sample(full_feature_pool, CONFIG["max_features"])
9     if len(full_feature_pool) > CONFIG["max_features"]
10    else full_feature_pool
11 )
12
13 # 计算积分图和特征矩阵
14 x_pos_train = compute_feature_matrix(integral_images(
15     pos_train_images), selected_features)
16 x_neg_train = compute_feature_matrix(integral_images(
17     neg_train_images), selected_features)
18 x_pos_val = compute_feature_matrix(integral_images(pos_val_images
19     ), selected_features)
20 x_neg_val = compute_feature_matrix(integral_images(neg_val_images
21     ), selected_features)
22
23 # 使用 AdaBoost 逐级训练级联分类器
```

```

2 stages = train_cascade(x_pos_train, x_neg_train, x_pos_val,
    x_neg_val, CONFIG)
3 val_pos_margins = cascade_margins_from_matrix(x_pos_val, stages)
4 val_neg_margins = cascade_margins_from_matrix(x_neg_val, stages)
5 final_threshold = calibrate_final_threshold(
6     val_pos_margins,
7     val_neg_margins,
8     CONFIG["final_min_recall"],
9 )
10 model = TrainedCascade(selected_features, stages, final_threshold
    )

```

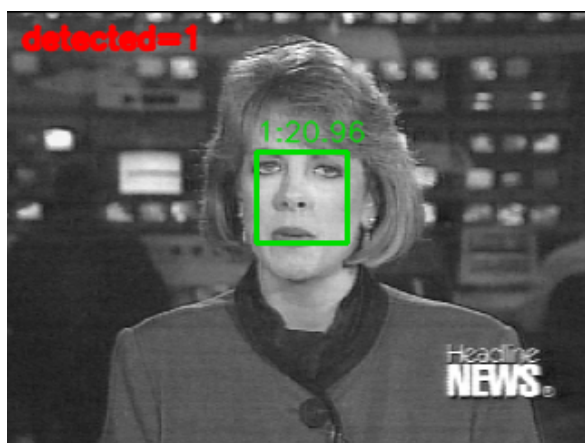
```

1 # 在测试图像上进行多尺度滑窗检测
2 for scale in generate_scales(gray.shape[0], gray.shape[1], cfg):
3     win = max(cfg["window_size"], int(round(cfg["window_size"] *
4         scale)))
5     step = max(4, int(round(win * cfg["sliding_window_step_ratio"]
6         )))
7     for y in range(0, gray.shape[0] - win + 1, step):
8         for x in range(0, gray.shape[1] - win + 1, step):
9             patch = gray[y : y + win, x : x + win]
10            patch = prepare_patch(patch, cfg["window_size"])
11            ii = integral_image(patch)
12            passed, score = cascade_predict(ii, model.features,
13                model.stages)
14            if passed and score >= model.final_threshold:
15                raw_detections.append((x, y, win, win, float(
16                    score)))
17
18 # 对检测结果进行聚类支持过滤和非极大值抑制
19 clustered = cluster_supported_detections(
20     raw_detections,
21     cfg["cluster_iou_threshold"],
22     cfg["min_cluster_support"],
23 )
24 detections = non_max_suppression(clustered, cfg["
    nms_iou_threshold"])

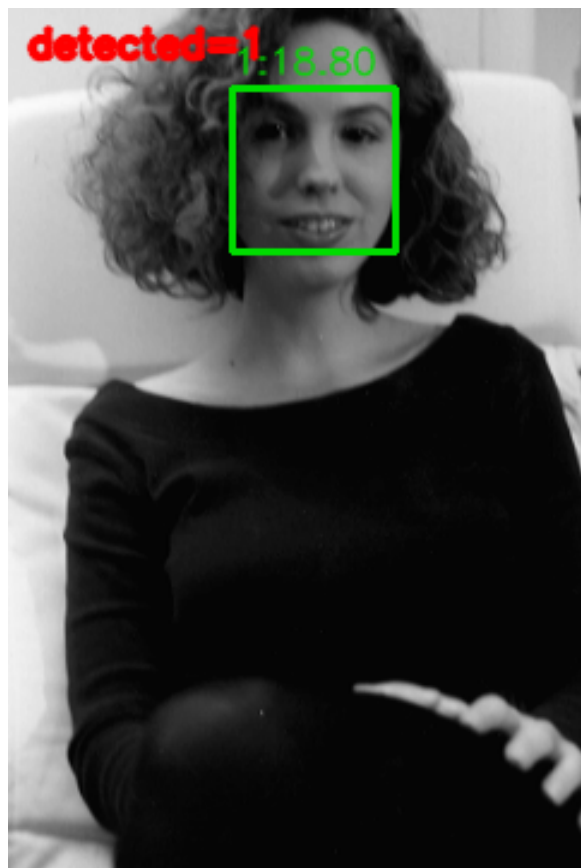
```

b) 实验结果截图

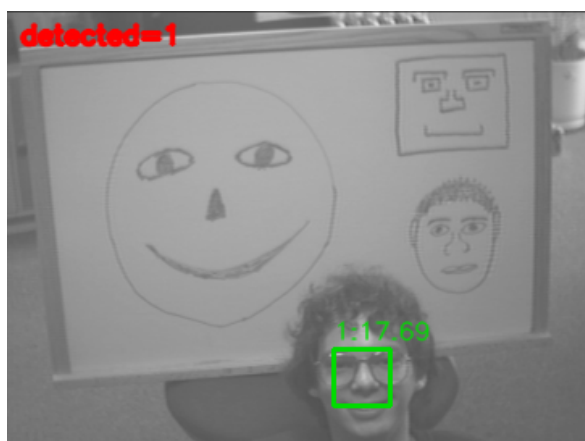
训练数据集为 MIT face/nonface patch dataset, 测试展示数据集为 CMU frontal_images (test、newtest、test-low)。



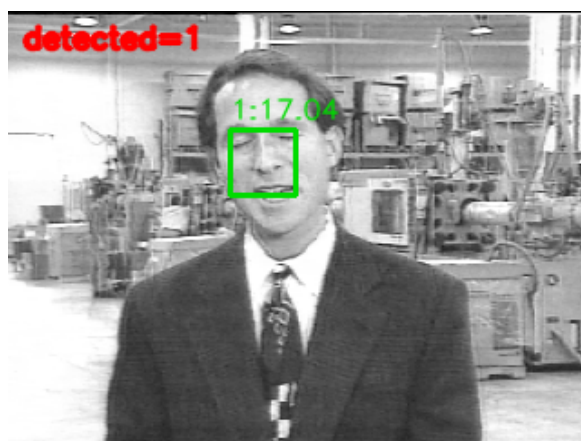
检测结果 1



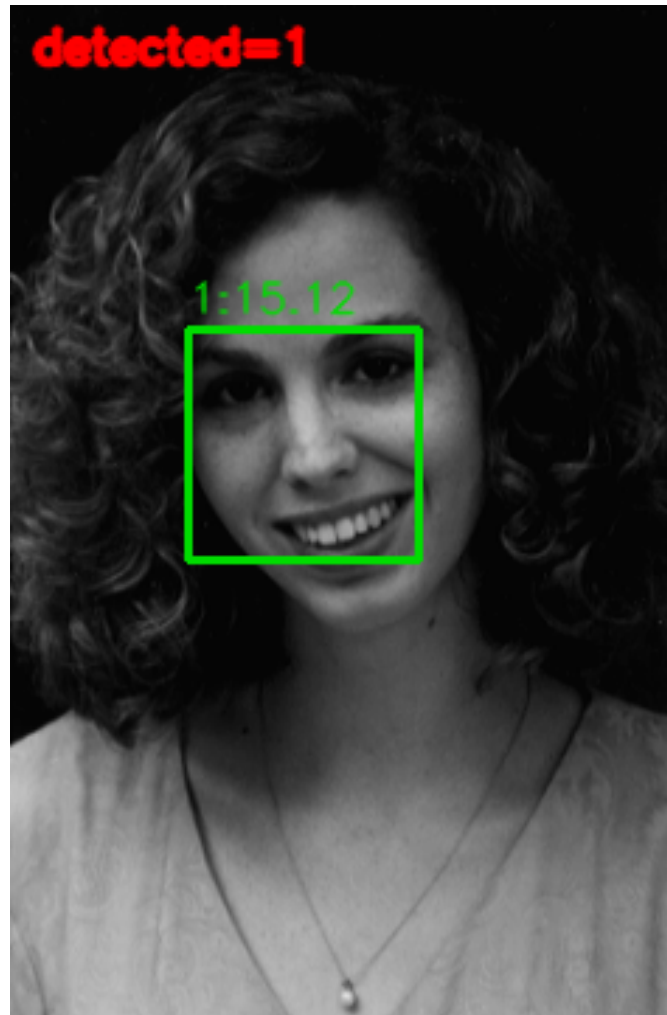
检测结果 2



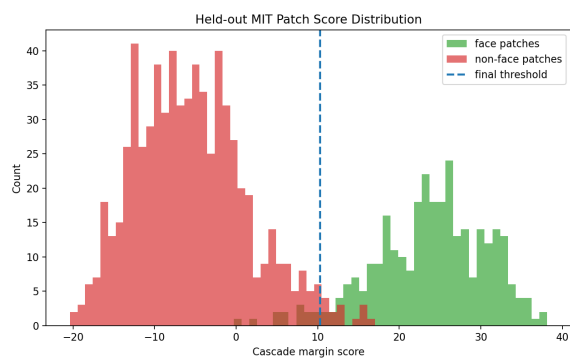
检测结果 3



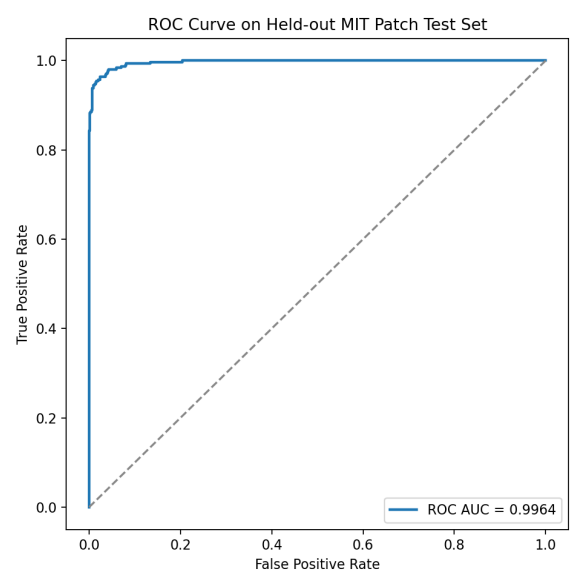
检测结果 4



检测结果 5



测试分数分布图



ROC 曲线

c) 实验小结

实验以 MIT face/nonface patch 数据集为基础，先计算积分图，再提取 Haar 特征，通过 AdaBoost 逐级训练强分类器，并将多个强分类器串联形成级联结构。训练好后应用到独立的 CMU frontal_images 数据集上，采用多尺度滑窗方式完成整图中的人脸检测。

最初检测的结果比较糟糕，图上很多空的框，降低对召回率的要求后，又容易出现漏检的情况。最终达到了一个平衡，检测结果展示在上面的检测图中。观察测试分数分布图反映了分数阈值，分类器算出的分数高于阈值便是人脸，低于阈值便是非人脸，底下交叉的部分是会检测错误的部分，可以看到大部分情况还是能检测正确的，ROC 曲线也反映了效果较好。不过真实的图片和专门用于训练的人脸非人脸还是有区别，依然很有可能出现漏检和误检。